协同进化算法

夏同

2026 年 3 月 16 日

目录

1 引言：协同进化算法基础 **4**
1.1 符号定义 4
1.2 算法引入与初期优势 4
1.3 基本流程（以 CCGA 为例） 4
1.4 问题可分解性定义 5

2 问题分解与变量分组策略 **6**
2.1 静态变量分组 6
2.2 随机变量分组 6
2.3 基于交互学习的变量分组 7
2.4 基于领域知识的变量分组 8
2.5 重叠与层次变量分组 8
2.6 混合变量分组 8

3 协作机制设计 **9**
3.1 协作者选择策略 9
3.2 个体适应度分配 10
3.3 子问题计算资源分配 11

4 算法实现与理论分析 **11**
4.1 在不同元启发式算法中的实现 11
4.2 并行实现 11
4.3 基准测试问题 12
4.4 理论分析 12
4.5 关键控制参数 12

5 协同进化算法的病态问题 **13**

6 承上启下章节 **13**
6.1 适应度评估的计算代价 14
6.2 差分分组的基本原理 14
6.3 四重挑战：驱动技术演进的根本动力 15

7 差分分组的高效改良与理论拓荒（2020-2022） **15**
7.1 高效自适应差分分组（EADG, 2022） 15
7.2 双差分分组（DDG, 2022） 17

8 面向复杂结构的分解技术：从平权到分层（2022-2024） **18**
8.1 重叠增强差分分组（OEDG, 2024） 19
8.2 基于贡献度的协同进化（CBCC-O, 2020） 21
8.3 分层差分分组（HDG, 2024） 23

9 学习型分解与古典融合的范式转移（2024-2026） **25**
9.1 基于学习的协同进化（LCC, 2025） 25
9.2 复合可分性分组（CSG, 2026） 27

10 其他值得关注的新方法与预研趋势（2023-2026） **29**
10.1 基于变量重要性的动态协同进化（DCCVI, 2025） 29
10.2 基于图神经网络的分解（GNN-based Decomposition） 29
10.3 时空图持续跟踪（Spatiotemporal Graph Continuous Tracking） 30
10.4 多臂老虎机资源感知调度（Multi-armed Bandit Resource-Aware Scheduling） . 30

11 横向对比与元分析 **30**
11.1 开销-精度权衡曲线 31
11.2 问题类型光谱的覆盖边界 31
11.3 假设的隐蔽性与脆弱性 31
11.4 理论保证与工程实效的割裂 33

12 未来展望 **33**
12.1 方向一：神经引导与符号推演的混合架构（Neuro-Symbolic Decomposition） . 33
12.2 方向二：针对动态重叠环境的时空图持续跟踪（Spatiotemporal Graph Continuous Tracking） 34
12.3 方向三：资源感知的自适应保真度分解（Resource-Aware Adaptive Fidelity Decomposition） 34
12.4 三个方向的协同关系 34

13 结语 **34**

1 引言：协同进化算法基础

1.1 符号定义

- $\mathbf{x} = (x_1, \ldots, x_n)$: n 维决策向量。
- Ω: n 维可行决策空间。
- Δx_i: 对变量 x_i 的微小扰动。
- $\Delta_i \mathrm{f}(\mathbf{x})$: 扰动变量 x_i 后，目标函数 f 的变化量。
- $\Delta_{ij} \mathrm{f}(\mathbf{x})$: 同时扰动变量 x_i 和 x_j 后，目标函数 f 的变化量。
- m: 子组件的数量。
- N: 子种群的规模。
- n: 决策变量的总数。
- $O_{i,j}$: 第 i 个子种群中的第 j 个个体。
- s: 子组件中的变量个数。
- $\mathbf{b}$: 上下文向量，由每个子种群的代表解组合成的完整问题解。
- 子组件 $\mathbf{x}_i$: 决策向量 $\mathbf{x}$ 的一个子集。
- 子种群: 用于优化相应子组件的个体群体。
- 子问题: 优化相应子组件的问题，即仅考虑 $\mathbf{x}$ 的子集来优化原始问题。

1.2 算法引入与初期优势

协同进化的灵感来源于生态学中的互利共生现象，即不同物种以互利的关系生活在一起。首个协同进化算法，即协同进化遗传算法，由 Potter 和 De Jong 于 1994 年提出。该算法因其在解决复杂优化问题上的潜力而激发了广泛的研究兴趣。协同进化算法之所以有效，关键在于其“分而治之”的策略。它将原始问题分解为一系列低维、易于处理的子问题。每个子问题由一个独立的子种群并行地求解。要评估某个子种群中一个个体的优劣，需要将该个体与其他子种群中选出的代表个体组合成一个完整的解，然后用这个完整解的质量来衡量该个体的贡献。简单来说，就是“你表现得好不好，要看你和队友合作得怎么样”。

与传统进化算法相比，协同进化算法的主要优势在于其分解策略，具体体现在四个方面：

- 并行化：问题分解天然适合并行计算，可以显著加速优化过程。
- 多样性保持：每个子问题由其独立的子种群求解，有助于维持种群的多样性。
- 鲁棒性与可重用性：将系统分解为模块化子问题，能提高对局部错误或故障的鲁棒性，也增强了在动态环境中的可重用性。
- 缓解“维度灾难”：如果分解得当，可以有效缓解随决策变量数量增加而导致性能急剧下降的问题。

1.3 基本流程（以 CCGA 为例）

最初的流程如下：

- **问题分解**：首先，将一个包含 n 个变量的优化问题，简单地分解为 n 个一维子问题（每个变量独立成为一个子问题）。
- **初始化**：为每个子问题随机初始化一个子种群。同时，创建一个“上下文向量”$\mathbf{b}$，它由从每个子种群中选出的代表个体（如当前最优个体）组合而成，代表当前找到的一个完整解。
- **协同进化（循环）**：算法循环地对每个子问题进行优化。
  - **子问题优化**：每个子问题由独立的遗传算法进行演化。在每一代中，为当前子种群 i 生成一批新个体（后代）。
  - **个体评估**：评估后代个体 $O_{i,j}$ 的适应度。这是协同进化的关键步骤。算法需要将该个体与其他子种群的代表组合成一个完整解，并用这个完整解的质量来衡量该个体的适应度。
    - **CCGA-1**：用该个体 $O_{i,j}$ 替换上下文向量 $\mathbf{b}$ 中对应子问题 i 的部分，形成一个新的完整解，然后用这个新解的目标函数值作为 $O_{i,j}$ 的适应度。
    - **CCGA-2**：生成多个完整解来评估。例如，除了用上下文向量 $\mathbf{b}$ 外，还可以从其他子种群中随机选择个体组成另一个完整解，取这两个完整解中较好的那个作为个体的适应度。
  - **更新上下文向量**：在所有新个体评估完毕后，检查是否有比当前上下文向量 $\mathbf{b}$ 更好的完整解，如果有，则更新 $\mathbf{b}$。
  - **选择**：基于父子代种群，通过适者生存的选择机制，构建新的子种群。

这个算法清晰地展示了协同进化的“分而治之”思想：原始问题被分解，各个子种群独立演化，而个体适应度的评估又强制了子种群间的协作。这种静态的一维分解在完全可分问题上效果显著，但在变量间存在复杂交互的不可分问题上则表现不佳。现实世界中的问题大多是不可分的，这促使研究者们探索更高级的问题分解策略。

1.4 问题可分解性定义

协同进化算法的效果与问题的可分解性密切相关。所以问题的可分解性必须要被良好的定义和解释。

- 完全可分问题：如果一个优化问题可以通过对每个决策变量进行独立的优化来求得全局最优解，那么它就是完全可分的。这类问题对协同进化算法来说相对容易，因为它可以将每个变量作为一个独立的子问题来处理。
- 完全不可分问题：如果问题中的任意两个决策变量都存在相互影响，则该问题是完全不可分的。
- k-不可分问题：介于完全可分和完全不可分之间，如果问题中最多有 k 个决策变量相互依赖，则称为 k-不可分问题。k 值越小，问题越容易分解和求解。
- 加性可分问题：这是文献中常用的一类 k-不可分问题。如果一个问题的目标函数可以表

示为多个互不重叠的子函数之和，那么它就是加性可分的:

$$f(\mathbf{x}) = \sum_{i=1}^{m} f_i(\mathbf{x}_i)$$

其中 $\mathbf{x}_1, \ldots, \mathbf{x}_m$ 是 $\mathbf{x}$ 的不相交子集，且每个子问题 f_i 只依赖于对应的子集。如果每个子集只包含一个变量，加性可分问题就退化为完全可分问题。加性可分问题比完全可分问题复杂，但只要将相互依赖的变量正确地划分到同一个子函数中，协同进化算法依然可以高效求解，因其结构清晰，常用于测试协同进化算法的性能。

讨论完可分解性，我们还要知道：理想的分解应基于变量间的相互作用，使得同一子组件内的变量高度相关，而不同子组件间的变量相互独立。如果相互作用的变量被错误地分到不同子组件中，算法很容易陷入“伪最小值”——由错误分解引入的虚假最优解。为此，研究者们提出了多种变量分组策略。

2 问题分解与变量分组策略

总的来说，问题分解是协同进化算法设计的首要和关键步骤。从简单的静态和随机分组，到能自动学习问题结构的交互式分组，再到利用领域知识和复杂结构的特殊分组，体现了该领域从“如何分解”向“如何更智能地分解”的演进。

2.1 静态变量分组

静态分组在优化开始前就预先设定好变量的分组方式，并在整个过程中保持不变。静态分组的局限性在于，它需要预先知道子组件的合适维度，这在处理未知问题时非常困难。

- **一维分解**：最初的协同进化遗传算法采用的正是这种方法，将每个变量视为一个独立的子问题。这种方法在完全可分问题上表现良好。
- **固定维度的多维分解**：为了处理一定程度的变量交互，可以将 n 维问题平均分解为 m 个 s 维的子问题（$n = m \cdot s$）。例如，将 1000 维的问题分解为 100 个 10 维的子问题。这种方法在某些不可分问题上优于一维分解。
- **顺序滑动窗口**：针对超大规模优化问题，研究者提出了一种顺序滑动窗口的静态分解方法，依次对相邻的变量进行分组优化。

2.2 随机变量分组

随机分组策略在每个协同进化周期开始前，都随机地将变量划分到不同的子组件中。这种做法的目的是增加相互作用的变量在同一个周期内被分到一起的概率。随机分组策略的局限性在于，如果相互作用的变量数量很大，将它们同时分到同一个子组件的概率仍然很低，尤其是在高维问题中。

- **固定子组件大小的随机分组**：预先设定子组件的大小 s，然后在每个周期随机地将 n 个变量分配到 $m = n/s$ 个互斥的子组件中。这种方法在不可分问题上通常优于静态分组，因为经过多个周期的随机组合，相互作用的变量有很大概率会在某个周期内被分到一起。
- **动态子组件大小的随机分组**：随机分组的一个难题是 s 的选择。如果 s 太小，无法捕捉变量交互；如果 s 太大，又失去了分解的意义。动态策略通过自适应地调整 s 来解决这个问题。例如，可以从一组预定义的大小 $\{s_1, s_2, \ldots, s_t\}$ 中，根据它们近期的优化表现来动态选择本轮使用的 s。

2.3 基于交互学习的变量分组

理想的情况是，算法能够自动学习问题中变量间的依赖关系，并基于此进行分组。这被称为链接学习或变量交互学习。基于交互学习的分组策略能够更精准地揭示问题的内在结构，从而大幅提升协同进化算法解决复杂不可分问题的能力。但其计算开销，特别是基于扰动和复杂模型的方法，是需要权衡的因素。主要方法包括：

- **基于扰动的方法**：这是最直观的方法，通过扰动一个或多个变量并观察目标函数值的变化来判断变量间是否存在交互。
  - **基本原理**：如果同时扰动两个变量带来的函数变化，不等于分别扰动它们带来的变化之和，即：
    $$\Delta_i f(\mathbf{x}) + \Delta_j f(\mathbf{x}) \neq \Delta_{i,j} f(\mathbf{x})$$
    那么，这两个变量很可能是相互作用的。因为一个变量的效应受到了另一个变量取值的影响。
  - **交互强度**：进一步地，可以用两边差值的绝对值来衡量交互的强度。
  - **扩展到连续优化**：上述原理也被扩展到连续优化问题。一种常用的判定准则是，如果存在一个解 $\mathbf{x}$ 和两个扰动，使得分别扰动一个变量的效果，与在另一个变量被扰动后再扰动该变量的效果不一致，则判定它们相互作用。数学上，这等价于两个扰动的效应乘积为负。
  - **代价与改进**：基于扰动的方法通常需要 $O(n^2)$ 次额外的函数评估来检测所有成对交互，代价较高。后续研究提出了全局差分分组等方法，旨在用更少的评估次数识别所有交互，包括直接和间接的交互。快速交互识别方法则尝试用固定次数的评估来判断一个变量是否与一组变量可分。
- **基于统计模型的方法**：这类方法利用种群进化过程中产生的历史数据，通过统计分析来推断变量间的依赖关系。常用的度量包括：
  - 变量间的皮尔逊相关系数、互信息等。
  - 变量与目标函数之间的相关性。
  - 变量值的分布，如方差、双变量联合分布与单变量分布的差异等。
- **基于分布模型的方法**：分布估计算法是这类方法的代表。它通过从有希望的解集中构建

概率模型来描述变量的分布和交互，然后基于这个模型采样生成新的解。从最初假设变量独立的单变量模型，到能捕捉成对交互的双变量模型，再到能处理高阶交互的贝叶斯网络，模型复杂度逐步提升，但计算代价也随之增加。

- **基于近似模型的方法**：对于评估代价昂贵的实际问题，可以用代理模型来近似目标函数。通过分析代理模型，如高维模型表示，可以揭示输入变量是如何独立或联合地影响输出的，从而识别变量交互。
- **链接自适应的方法**：这类方法在进化过程中动态地调整变量的分组方式。例如，可以在个体的染色体编码中附加一些标志位，这些标志位指示了基因之间的“链接”关系，并在进化的过程中通过遗传操作来学习和优化这些链接，从而将相互作用的基因自动组合在一起。

2.4 基于领域知识的变量分组

如果对优化问题有深入的先验知识，可以直接利用这些知识来指导变量分组，这往往是最有效的方法。例如：

- 在电力系统优化中，可以根据无功功率和电压的局部特性，将电网划分为几个相对独立的子系统。
- 在飞行冲突避免问题中，可以根据两架飞机的冲突概率来判断它们之间的交互强度，并据此分组。
- 在多仓库车辆路径问题中，可以根据客户与仓库的邻近关系，将问题分解为多个单仓库的子问题。

2.5 重叠与层次变量分组

现实世界中的变量交互可能非常复杂，简单的非重叠分组不足以捕捉其全貌。

- **重叠分组**：在这种策略中，一个变量可以同时属于多个子组件。这为解决完全不可分问题提供了新思路，但同时也带来了新挑战：如何设计有效的重叠分组？如何为个体评估构建完整的解？共享变量如何在不同的子组件间协调？
- **层次分组**：变量间的交互可能具有层次结构，即低层次变量的组合作为一个整体，在高层次上与其它变量发生交互。层次分组策略旨在通过逐层分解和组合来匹配这种结构，例如，先将强相关的变量聚合成一个“构建块”，再将这些构建块作为新的变量进行更高层次的优化。

2.6 混合变量分组

没有任何一种分组策略是万能的。混合分组策略将多种方法结合起来，以期取长补短。例如，可以将静态的一维分解与随机分组相结合，或者在基于扰动的方法中融入概率分布模型的信息。

3 协作机制设计

除了问题分解，子问题之间如何协作也是决定算法性能的关键。个体适应度评估需要来自其他子种群的“协作者”来构建完整解。选择哪些协作者、选择多少协作者，直接影响着评估的准确性和算法的搜索行为。这主要涉及两个因素：选择压力（选择的贪婪程度）和协作者池大小（用于评估的完整解数量）。

3.1 协作者选择策略

3.1.1 用于单目标优化问题的协作者选择策略

对于只有一个目标函数的优化问题，协作者选择策略旨在构建一个或多个完整解来评估个体适应度。

- **1. 构建单个完整解的策略：**
  - **单一最优协作者选择：**这是最常用、最贪婪的策略。评估个体时，总是将它与其它子种群当前的最优个体（存储在上下文向量中）组合。这种方法收敛快，但容易陷入局部最优，尤其在不可分问题上。
  - **单一最差协作者选择：**与最优协作者相反，它总是选择最差的个体作为协作者，提供了一种极端的评估视角。
  - **随机协作者选择：**随机从其他子种群中选择个体作为协作者。这种方法有助于维持种群多样性，防止早熟收敛，但可能会减慢收敛速度。
  - **精英协作者选择：**从每个子种群中选出 K 个最优个体组成一个协作者池，然后从中随机选择协作者。K 是一个可调参数，$K = 1$ 时等同于单一最优选择，$K = N$（种群规模）时等同于随机选择。可以通过动态调整 K 来平衡探索和开发。
  - **基于轮盘赌/锦标赛的协作者选择：**根据个体的适应度，通过轮盘赌或锦标赛等选择机制，从其他子种群中选择协作者。这给了优秀个体更多作为协作者的机会。
  - **链接协作者选择：**将各个子种群中位置相同或相关联的个体组合在一起形成完整解。这种方式可以减小由于协作者变化带来的系统波动。
  - **基于邻域的协作者选择：**个体只与其它子种群中与其“相邻”的个体进行合作，例如在种群线性排列中的前后邻居。
- **2. 构建多个完整解的策略：**
  - **完整协作者选择：**个体将与来自其他子种群的所有可能个体组合进行合作，形成 $N^{(m-1)}$ 个完整解，并取其中的最佳适应度作为该个体的适应度。这保证了评估的全面性，理论上可以收敛到全局最优，但计算代价极高。
  - **基于存档的协作者选择：**维护一个记录优秀协作者的存档，个体只与存档中的协作者组合。存档可以是帕累托前沿的非支配解，或者是那些能够有效区分个体优劣的“信息性”协作者。这可以大大减少评估次数。
  - **混合协作者选择：**结合多种策略。例如，经典的“最优 + 随机”策略，先用最优

协作者构建一个解，再与几个随机协作者构建解，取其中最佳作为适应度。这种方法简单有效，兼顾了贪婪性和多样性。

  - **自适应协作者选择**：协作者数量或选择策略可以随时间变化。例如，早期使用更多协作者以充分探索，后期减少协作者以加速收敛。

3.1.2 用于多目标优化问题的协作者选择策略

对于多目标优化问题，协作者的选择还需要考虑解的收敛性和多样性。协作者选择策略是连接各个子种群的桥梁。选择合适的策略，可以在评估精度、计算效率和搜索行为之间取得良好平衡，对于算法的成功至关重要。除了可以沿用单目标中的一些策略外，还发展出一些专门的方法：

- **基于非支配解的方法**：从其他子种群的帕累托非支配解集中随机选择协作者。这样形成的完整解本身就是非支配解，有助于推动整个种群向帕累托前沿前进。
- **基于偏好的方法**：从非支配解集中，根据某种偏好准则选择协作者。例如，选择对种群多样性贡献最大的“膝点”解，或者对超体积指标贡献最大的解。这旨在引导搜索向帕累托前沿中更具价值的区域进行。
- **基于竞争的方法**：当变量分组存在重叠时，持有相同变量的子种群之间会形成竞争关系，表现更好的子种群有更高的概率来决定共享变量的取值。

3.2 个体适应度分配

个体的适应度反映了它与其他子种群协作构建完整解的能力。根据构建的完整解数量，适应度分配主要有以下几种方式：

- **基于单个完整解**：直接使用该个体参与构建的单个完整解的适应度值作为其适应度。这种方法简单高效，但评估结果可能具有偏差，因为该个体与其他协作者的配合效果未能完全体现。
- **基于多个完整解**：让个体参与构建多个完整解，然后将这些解的适应度值聚合起来作为其最终适应度。聚合方式可以是取最大值、取平均值、取最小值，或者使用锦标赛选择等方式。取最大值是乐观的，鼓励个体在最佳配合下的表现；取平均值则更稳健，反映个体的平均协作能力。
- **其他方法**：
  - **帕累托支配**：在多目标优化中，可以根据个体参与构建的解之间的帕累托支配关系来分配适应度。
  - **适应度继承**：为了节省计算资源，部分后代的适应度可以直接从其父代适应度加权平均得到，而无需进行完整的协作评估。
  - **启发式估计**：在训练神经网络等特定应用中，可以根据神经元的权重、网络输出的投票数等启发式信息来估计其对整体性能的贡献。

3.3 子问题计算资源分配

由于适应度评估通常消耗绝大部分计算资源，如何合理地将这些资源分配给不同的子问题至关重要。如果所有子问题同等重要，传统的循环策略（每个子问题依次获得相同资源）是可行的。但实际问题中，子问题的难度和对最终解的贡献往往是不平衡的。因此，将更多资源投入到更有“前途”或贡献更大的子问题上更为合理。

- **基于长期贡献的方法：**根据子问题在整个优化历史中对改进完整解的累积贡献来分配资源。贡献大的子问题将获得更多优化机会。
- **基于近期贡献的方法：**子问题的贡献可能随时间变化。因此，可以根据它在最近几代中的表现来动态分配资源，将资源聚焦于当前最有效的子问题上。
- **基于不同子种群规模的方法：**为不同子问题分配不同规模的子种群，也是一种隐式的资源分配方式。贡献潜力大的子问题可以使用更大的种群进行更细致的搜索。

个体适应度分配和子问题资源分配共同决定了协同进化算法的效率和效果。前者关乎评估的准确性，后者关乎资源使用的经济性。

4 算法实现与理论分析

协同进化是一种框架思想，可以集成到多种具体的元启发式算法中，并且天然适合并行计算。

4.1 在不同元启发式算法中的实现

协同进化框架的灵活性使其能与多种算法结合：

- **遗传算法：**这是最早也是最常见的结合方式，协同进化遗传算法是其他所有变体的基础。
- **粒子群优化：**将粒子群分成多个子群，每个子群负责优化一部分变量，子群间通过共享信息（如全局最优位置）来协作。这种方法在连续优化中应用广泛。
- **差分进化：**同样在连续优化中表现优异，其变异和交叉操作可以与协同进化的分解-协作框架很好地融合。
- **蚁群优化：**更擅长解决组合优化问题，如路径规划。可以将问题分解为多个子路径，由不同的蚁群分别搜索，再通过信息素交换进行协作。
- **遗传编程：**常用于程序或模型的自动生成。协同进化可以促进任务的分解，例如，让一个种群进化主程序，另一个种群进化子程序或参数。
- **其他算法：**协同进化也与进化策略、人工蜂群、人工免疫系统、代理模型辅助搜索、模因算法等进行了结合，形成了各具特色的混合算法。

4.2 并行实现

“分而治之”的思想使协同进化算法天生就适合并行计算。并行实现极大地提升了协同进化算法处理大规模和复杂优化问题的能力。其并行化主要有以下几个层次：

- **子问题级并行**：将不同的子问题及其对应的子种群分配到不同的计算节点上（如 PC 集群、GPU 核心），各个节点独立演化自己的子种群，只在需要协作评估个体适应度时才进行通信。
- **种群内并行**：对于单个子种群内的操作，如适应度评估、交叉变异等，也可以进行并行化处理。
- **混合并行**：结合上述两种方式，实现更细粒度的并行。

为了科学地评估和比较不同的协同进化算法，需要一套标准的测试问题和严谨的理论分析工具，同时也要关注算法关键参数的设置。

4.3 基准测试问题

一个好的基准测试问题集应该包含各种特征和难度级别，以便全面评估算法的性能。

- **早期函数**：如 Rastrigin、Schwefel、Griewangk 和 Ackley 等经典测试函数，它们具有不同的多峰、不可分特性，被最早用于测试协同进化算法。
- **伪布尔问题**：为了简化理论分析，一系列二进制问题被提出，如 OneMax（简单可分）、LeadingOnes（变量间存在顺序依赖）、Trap（存在欺骗性吸引子）等。
- **大规模优化问题**：随着问题规模的增大，一系列针对大规模全局优化的基准测试问题集被提出，如 CEC'2010 和 CEC'2013 的大规模优化问题竞赛中使用的函数，它们通常包含各种复杂的分量交互和病态条件。

4.4 理论分析

虽然大多数研究通过实验验证算法性能，但理论分析能更深入地揭示算法的内在工作原理和收敛性保证。

- **离散优化分析**：研究者利用期望优化时间（即首次找到全局最优解所需的平均函数评估次数）来分析协同进化算法在伪布尔问题上的表现。例如，证明了在可分问题 OneMax 上，协同进化算法可以达到与标准进化算法相当甚至更好的复杂度。但也指出了在 Trap 等具有欺骗性的问题上，简单的协同进化算法可能以高概率失败。
- **连续优化分析**：协同进化算法与数学规划中的块坐标下降法有深刻联系。后者的理论成果，如在某些条件下可以线性收敛到局部最小值、全局收敛性等，可以为理解协同进化算法的行为提供理论基础。此外，对于加性可分问题，研究者也为识别变量交互提供了理论依据。

4.5 关键控制参数

协同进化算法的性能受几个关键参数的影响：

- **子组件数量**：这是最重要的参数，通常由问题分解策略决定。在静态和随机分组中，它是预设的；在基于交互学习的分组中，它由学习到的依赖关系动态确定。

- **子种群规模**：每个子种群的大小。不同子问题使用不同种群规模的研究相对较少，这是一个值得探索的方向。
- **计算资源分配策略**：如 CBCC 算法中的贡献度计算周期、多臂老虎机选择中的平衡因子等，这些参数控制着资源分配的动态行为。

5 协同进化算法的病态问题

由于个体的适应度依赖于变化的协作者，协同进化算法可能遭遇一些传统进化算法中不存在的特殊问题。

- **红皇后效应**：描述了一种现象，即一个子种群的个体变得“更好”，仅仅是因为它的协作者变差了，而不是它自身有真正的进步。这导致种群的进化更像是一场原地踏步的“军备竞赛”，而非向全局最优的持续逼近，表现为适应度值的振荡。
- **相对过度泛化**：算法可能倾向于收敛到一个“稳健”但并非最优的解。例如，在有两个山峰的问题中，一个山峰高而窄，另一个山峰低而宽。与窄峰上的“专家”合作，只有特定组合才能获得高分；而与宽峰上的“通才”合作，无论与谁搭档都能获得不错的分数。在评估时，由于通才频繁被选为协作者，那些能与通才很好配合的个体会获得持续的高评价，从而使整个搜索偏向于宽峰，而错过了真正的最优解。
- **梯度丧失**：当一个子种群失去多样性后，其他子种群的适应度景观就会变得“平坦”，因为无论它们怎么变化，组合出来的解都因协作者的单一而无法产生显著差异。这导致所有子种群都失去进化动力，陷入停滞。

理解这些病态问题对于设计更鲁棒的协同进化算法至关重要，研究者们也提出了如帕累托存档、信息性协作者、新颖性搜索等方法来应对这些问题。

6 承上启下章节

协同进化算法通过“分而治之”的策略处理大规模优化问题，其核心在于如何有效地将高维问题分解为若干低维子问题。本文系统梳理了 2020 年至 2026 年间协同进化算法在问题分解领域的关键进展。我们将这七年的演进划分为三个阶段：差分分组的高效改良与理论拓荒（2020-2022）、面向复杂结构的分解技术突破（2022-2024）、以及学习型分解与古典融合的范式转移（2024-2026）。通过对 EADG、DDG、OEDG、CBCC-O、HDG、LCC、CSG 等代表性算法的深入剖析，揭示了其在分组精度、计算效率与问题结构适应性之间的根本性权衡。在此基础上，本文进行了横向元分析，识别了各类算法隐含的假设脆弱性与适用范围边界。最后，我们展望了神经符号混合架构、时空图持续跟踪、自适应保真度分解三个有望突破当前困境的未来方向，并指出可解释性危机将是定义下一代协同进化算法研究议程的核心议题。

缩写	全称	中文解释
CC	Cooperative Co-evolution	协同进化
LSGO	Large-Scale Global Optimization	大规模全局优化
BBO	Black-Box Optimization	黑盒优化
FE	Fitness Evaluation	适应度评估
DG	Differential Grouping	差分分组
EADG	Efficient Adaptive Differential Grouping	高效自适应差分分组
DDG	Dual Differential Grouping	双差分分组
OEDG	Overlapping Enhanced Differential Grouping	重叠增强差分分组
CBCC-O	Contribution-Based Cooperative Co-evolution for Overlapping problems	基于贡献度的协同进化（处理重叠）
HDG	Hierarchical Differential Grouping	分层差分分组
LCC	Learning-based Cooperative Coevolution	基于学习的协同进化
CSG	Composite Separability Grouping	复合可分性分组
MDP	Markov Decision Process	马尔可夫决策过程
PPO	Proximal Policy Optimization	近端策略优化
GNN	Graph Neural Network	图神经网络
STGNN	Spatiotemporal Graph Neural Network	时空图神经网络
OOD	Out-of-Distribution	分布外

6.1 适应度评估的计算代价

适应度评估（Fitness Evaluation, FE）是优化过程中计算目标函数的单次操作。在工程实践中，一次评估可能对应数小时的 CFD 仿真或大规模数值模拟，因此 FE 次数成为衡量算法效率的核心指标。经典差分分组（Differential Grouping, DG）需要 $O(n^2)$ 次评估来检测所有成对交互，当 $n = 10^6$ 时，10^{12} 次评估即使以每秒一亿次的速度进行，也需要一万秒以上，且这仅完成分解阶段。如何降低分解开销，成为算法设计的首要约束。

6.2 差分分组的基本原理

在协同进化算法中，“分组”指的是将决策变量划分到不同子问题的过程。最简单的分组方式包括静态分组（优化前预先固定分组，如按变量下标连续划分）和随机分组（每轮进化随机重新划分变量）。然而，这些方法要么需要先验知识，要么无法适应变量间复杂的依赖关系，容易导致交互变量被错误拆分。

差分分组（Differential Grouping, DG）应运而生，它通过数值扰动自动检测变量间的交互，从而指导分组。其核心思想是：如果两个变量相互独立，那么一个变量的变化对目标函数的影响不应随另一个变量取值的变化而改变。具体地，对于变量 x_i 和 x_j，在点 (x, y) 处进行四点探测：

- $f(x+\Delta x, y)$
- $f(x, y+\Delta y)$
- $f(x+\Delta x, y+\Delta y)$
- $f(x, y)$

定义差异指标：

$$\Delta_{x,y} = |[f(x+\Delta x, y) - f(x, y)] - [f(x+\Delta x, y+\Delta y) - f(x, y+\Delta y)]|$$

这个公式的本质是检验：变量 x_i 的改变所造成的影响，是否会因为变量 x_j 的状态不同而不同。若 $\Delta_{x,y}$ 小于阈值，则认为 x_i 与 x_j 独立，否则存在交互——即两者“串通一气”。该原理构成了后续众多分解算法的基础。基础的 DG 需要遍历所有变量对，因此评估次数为 $O(n^2)$，这在大规模问题中难以承受。

6.3 四重挑战：驱动技术演进的根本动力

2019 年之前的传统分解方法（包括静态分组、随机分组以及基础 DG）在现代工程问题面前暴露了四个难以逾越的鸿沟：

1. **规模挑战**：航空航天设计、神经网络架构搜索等场景决策变量高达百万级，$O(n^2)$ 复杂度无法承受。
2. **结构挑战**：变量间存在深度重叠（一个变量属于多个子系统）或层次依赖（核心变量与边缘变量），传统平权分解束手无策。
3. **评估挑战**：昂贵仿真一次数小时，任何需要大量前置评估的分解策略均不可行。
4. **先验挑战**：真实黑盒问题无梯度、无结构信息，传统依赖人工规则的分解策略失效。

正是这四重挑战，催生了 2020-2026 年百花齐放的分解技术。面对这四重挑战，研究者们并非一蹴而就。从最初对经典方法的高效化改良，到后来不得不直面复杂结构进行硬核攻坚，再到如今借助机器学习实现范式颠覆，这一演进路径清晰地勾勒出我们对大规模优化问题认知的深化过程。本文将按照技术演进的时间线与逻辑脉络，依次剖析各阶段代表性算法的设计思想、操作细节、理论支撑与适用范围。

7 差分分组的高效改良与理论拓荒（2020-2022）

2020 至 2022 年间，研究者主要致力于解决 DG 系列算法的计算效率瓶颈，同时尝试突破其仅能处理加性结构的理论局限。

7.1 高效自适应差分分组（EADG, 2022）

EADG（Efficient Adaptive Differential Grouping）由 Sun 等人在 2022 年提出，旨在将分解复杂度从 $O(n^2)$ 降低至 $O(n \log n)$，从而首次使协同进化算法能够以合理代价触及百万级变量问题。

7.1.1 设计思想与技术突破

EADG 的核心创新体现在三个层面：

1. **实例类型预检**：在全面分解前，通过少量随机采样判断问题是否完全不可分。若所有采样子集均表现出强耦合，则直接采用随机分组或一维分解，避免无效的精细探测。
2. **记忆化二叉树搜索**：将所有变量映射为一棵二叉树，根节点代表变量全集，逐层二分直至叶子节点（单变量）。算法从根开始，检测左右子节点（变量子集）间的交互性。根据差分定理的传递性，若两个子集相互独立，则它们包含的所有变量对也必然独立，从而无需进一步探测。这种“解复用”机制大幅减少了评估次数。打个比方：*EADG* 的二叉树搜索，就像警察排查一栋楼的嫌疑人——如果通过监控确认整个楼层的人都没作案时间，就无需再对每个房间进行逐一盘问，从而大大提高了效率。
3. **自适应阈值生成**：传统 DG 需要人工设定交互判定阈值，EADG 在探测过程中收集所有交互强度响应值，构建序数分布并动态生成归一化阈值，摆脱了对专家经验的依赖。

7.1.2 操作流程

1. **预检阶段**：随机抽取若干对变量子集进行基础差分探测。若所有探测均显示强交互，算法终止精细分解流程，转入随机分组模式。
2. **二叉树构建**：将 n 个变量构建为深度为 $\lceil \log_2 n \rceil$ 的完全二叉树，叶子节点对应单变量。
3. **自顶向下探测**：从根节点开始，对每个内部节点，检测其左右子节点对应的变量子集是否存在交互。检测方法为：在多个采样点上计算两组变量的联合扰动效应与独立扰动效应之和的差异。
4. **解复用与剪枝**：若某内部节点判定左右子集独立，则其所有后代节点之间的交互关系均被直接标记为独立，不再进行实际探测。
5. **阈值自适应**：实时记录所有实际探测得到的交互强度值，每完成一定数量探测后，更新序数分布并调整判定阈值。

7.1.3 理论支撑

EADG 的复杂度降低源于二叉树结构对搜索空间的剪枝。其核心逻辑基于差分定理的一条重要性质：变量间的交互具有“传递性”——如果两个变量子集整体上不存在交互，那么这两个子集中的任意单个变量之间也必然不存在交互。

形式化地，设子集 A 和 B 是决策变量的两个不相交集合。我们用 $\Delta_A f(\mathbf{x})$ 表示在点 $\mathbf{x}$ 处同时扰动 A 中所有变量（每个变量施加微小变化）所引起的函数值变化量；类似定义 $\Delta_B f(\mathbf{x})$ 和 $\Delta_{A\cup B} f(\mathbf{x})$（同时扰动 A 和 B 中所有变量）。若对于任意采样点 $\mathbf{x}$，都有

$$\Delta_{A\cup B} f(\mathbf{x}) = \Delta_A f(\mathbf{x}) + \Delta_B f(\mathbf{x}),$$

则称子集 A 与 B 之间无交互（即它们对函数的影响是可叠加的）。此时，可以严格证明：对于任意 $a \in A$ 和 $b \in B$，变量 a 与 b 之间也无交互，即 $\partial^2 f/\partial x_a \partial x_b \equiv 0$。

这一传递性使得 EADG 能够通过检测较大子集间的独立性来推断其内部所有细粒度变量对的独立性，从而避免了对所有变量对进行逐一探测。在二叉树结构中，若某内部节点（对应变量子集）的左右子节点被判定为独立，则该节点以下所有跨子树的变量对均可直接标记为独立，无需消耗任何实际评估。

因此，EADG 的最坏情况探测次数仅与二叉树内部节点数成正比（$O(n)$），再考虑到每个节点需进行常数次采样，总复杂度为 $O(n \log n)$，远低于经典 DG 的 $O(n^2)$。

7.1.4 局限性与适用范围

EADG 的主要局限在于：对弱间接交互敏感，在强噪声或多模态景观（指函数存在多个优劣不同的局部最优点，算法很容易陷入其中一个“陷阱”而无法自拔）中，微弱但关键的耦合可能被误判为独立；误差可能沿二叉树结构自顶向下呈“雪崩式传播”。适用于大规模加性可分或部分可分、适应度景观相对平滑的优化问题。

7.2 双差分分组（DDG, 2022）

DDG（Dual Differential Grouping）由 Li 等人于 2022 年提出，旨在将差分探测的适用范围从加性交互扩展到乘性交互，且不增加额外评估开销。

7.2.1 设计思想与技术突破

在理解 DDG 之前，我们需要明确两种交互类型：

- **加性交互**：指变量间存在非加性依赖，即函数不能分解为各子函数之和。经典 DG 通过检测二阶差分来识别这种交互——若两个变量独立，则它们对函数的影响可叠加；否则存在加性交互。
- **乘性交互**：指变量间存在乘性依赖，即函数可写成乘积形式 $f(x, y) = g(x)h(y)$。这类函数在原始空间中变量是耦合的（因为一个变量的效果依赖于另一个变量的取值），但通过对数变换可转化为加性形式。

DDG 的核心洞察在于对数变换的**同态性质**：对数运算将乘法结构映射为加法结构——就像 $\log(ab) = \log a + \log b$ 一样，它把乘积关系“翻译”成加法关系。因此，对于乘性可分函数 $f(x, y) = g(x)h(y)$，取对数后得到 $\ln f(x, y) = \ln g(x) + \ln h(y)$，恰好变为加性形式。这样，我们就可以用经典的差分探测方法来检测对数空间中的加性独立性，从而间接识别原始空间中的乘性交互。

关键在于，对数变换所需的函数值完全复用加性探测的物理采样点（即四点探测得到的 f 值），因此无需进行额外的适应度评估，仅增加简单的对数计算即可获得乘性交互信息。

7.2.2 操作流程

1. **四点探测**：与经典 DG 相同，获取四个点的函数值 $f(a, b), f(c, b), f(a, d), f(c, d)$。

2. **加性差异指标**：计算标准 DG 的差异量

$$\Delta_{addi} = |[f(a,b) - f(c,b)] - [f(a,d) - f(c,d)]|$$

若 $\Delta_{addi} \leq \epsilon_{addi}$（$\epsilon_{addi}$ 为加性判定阈值），则初步判定两变量在加性层面独立。

3. **乘性差异指标**：对同一组探测值取对数，再计算类似差异

$$\Delta_{multi} = |[\ln f(a,b) - \ln f(c,b)] - [\ln f(a,d) - \ln f(c,d)]|$$

4. **双重判定**：若 $\Delta_{multi} > \epsilon_{multi}$ 且 $\Delta_{addi} > \epsilon_{addi}$，则判定这两个变量之间存在强烈的乘性耦合结构（即它们在原始空间中不可加性分离，但通过对数变换后可能变为加性可分）。

7.2.3 理论支撑

DDG 的数学基础源于对数变换的代数性质。设 $f(x,y) = g(x)h(y)$ 是乘性可分的，则：

$$\ln f(x,y) = \ln g(x) + \ln h(y)$$

在 $\ln f$ 空间中，函数变为加性可分，因此其二阶交叉偏导为零。这意味着对于任意四个点 $(a,b),(c,b),(a,d),(c,d)$，差分探测结果应满足：

$$[\ln f(a,b) - \ln f(c,b)] - [\ln f(a,d) - \ln f(c,d)] = 0$$

即 $\Delta_{multi} = 0$。反之，如果 f 不是乘性可分的（即存在更复杂的耦合），则 $\ln f$ 空间中通常会出现非零的 Δ_{multi}。因此，通过检测 Δ_{multi} 是否超过阈值，可以判断是否存在乘性交互。

注意，Δ_{multi} 的计算完全复用了加性探测的物理采样点——四个 f 值已经通过评估获得，后续只涉及对数运算，没有引入新的适应度评估。因此，DDG 在保持与经典 DG 相同评估开销的前提下，将可处理的问题类型从加性交互扩展到了乘性交互。

7.2.4 局限性与适用范围

DDG 面临严峻的数值稳定性挑战：

- **定义域限制**：对数变换要求函数值严格为正，这在许多实际问题中不成立。
- **数值下溢**：当函数值接近零时，对数趋近于负无穷，导致浮点运算不稳定。
- **混合结构模糊**：当问题同时包含加性与乘性交互时，双重判定可能产生歧义（例如，Δ_{addi} 和 Δ_{multi} 均超阈值，但无法区分是纯乘性耦合还是混合型）。

因此，DDG 适用于具有纯加性或纯乘性结构、且适应度定义域经过严格非负预处理的数学与工程模型。

8 面向复杂结构的分解技术：从平权到分层（2022-2024）

前文介绍的 EADG 和 DDG 等算法虽然显著提升了分解效率或扩展了交互类型，但它们都隐含着一个共同假设：变量之间是“平权”的——即每个变量只属于一个子组件，子组件之

间互不相交，且所有变量在分组中地位平等。这种“平权分组”范式在处理完全可分或加性可分问题时表现良好，但面对真实世界的复杂系统则力不从心。

随着研究深入，学者们意识到许多工程问题的变量关系远非平权所能刻画，主要表现为两种复杂结构：

- **重叠结构**：一个变量可能同时属于多个子系统。例如，在机械设计中，齿轮的模数既影响传动效率（子问题 1），又影响制造成本（子问题 2）。若强行将此类“共享变量”划归单一子组件，必然导致其他子组件的评估失真，引发“相对过度泛化”等问题。
- **层次结构**：变量之间存在主从控制关系。例如，在航空发动机设计中，压气机级数（核心变量）决定了整体构型，而叶片倾角（边缘变量）仅作局部微调。若将核心变量与边缘变量等同视之，不仅浪费计算资源，更可能掩盖问题的本质结构。

2022 至 2024 年间，一系列针对上述复杂结构的新算法应运而生，标志着协同进化分解技术从“平权”走向“分层”与“重叠感知”。

8.1 重叠增强差分分组（OEDG, 2024）

在深入 OEDG 之前，我们先明确什么是“重叠变量”。考虑一个机械设计问题：齿轮的模数（记为变量 x）同时影响传动效率（子问题 1 的目标 f_1）和制造成本（子问题 2 的目标 f_2）。在协同进化框架中，如果我们简单地将 x 划归给子问题 1，那么评估子问题 2 的个体时，x 的取值将被固定（来自上下文向量），这会导致子问题 2 的适应度评估失真，因为 x 本应随子问题 2 的进化而调整。这种一个变量同时属于多个子问题的现象称为**重叠结构**。传统平权分组无法处理重叠，往往导致算法陷入“相对过度泛化”的陷阱。

OEDG（Overlapping Enhanced Differential Grouping）由 Wang 等人于 2024 年提出，旨在通过两阶段框架精确还原变量间的重叠拓扑结构，即识别出哪些变量是“共享”的，以及它们分别属于哪些子组件。

8.1.1 设计思想与技术突破

OEDG 采用“先粗后细”的两阶段策略，以平衡探测开销与精度：

1. **第一阶段：初步扫描（粗粒度分组）**：利用基础差分探测快速扫描所有变量，识别出可能存在交互的变量群体，形成若干粗糙的子组件，并标记出疑似重叠的共享变量。此阶段目标是尽可能多地发现潜在交互，允许将一些间接相关的变量暂时合并，形成“候选联合体”。
2. **第二阶段：微观精炼（细粒度拆分）**：对每个粗糙子组件进行深入分析，区分其中的**直接交互**与**间接交互**。所谓直接交互，是指两个变量直接耦合（例如 $f = x_1x_2$）；间接交互则指两个变量通过第三个变量产生关联（例如 $f = x_1x_3 + x_2x_3$，则 x_1 与 x_2 通过 x_3 间接相关）。间接交互往往导致变量被错误地归入同一子组件（形成**伪联合体**），OEDG 通过高阶分析拆解这些伪联合体，并将真正的重叠变量精确分配到其所属的多个子组件中。

为了实现上述目标，OEDG 引入了两个关键算子：

- **子组件联合检测（Subgroup Union Detection, SUD）**：该算子通过构建粗糙子组件内部的“交互强度矩阵”，分析变量之间的高阶依赖关系。如果发现某组变量之间缺乏直接的强交互（例如，它们两两之间的二阶差分很小，但通过第三变量的间接效应却显著），则判定它们属于伪联合体，并触发拆解。
- **子组件检测（Subgroup Detection, SD）**：在 SUD 拆解后，SD 算子对疑似重叠的变量进行更密集的局部扰动校验（例如缩小扰动步长、增加采样点），以精确判定每个重叠变量究竟应与哪些子组件关联，最终形成不交叠但有共享变量的分组结构。

8.1.2 操作流程

1. **初始化**：建立三个集合——疑似重叠变量集 OV（初始为空）、初步子组件集 N（初始为空）、未分组变量池 V_1（包含所有变量）。
2. **第一阶段循环**：
   - 从 V_1 随机选择一个变量作为基准，利用基础差分探测（INTERACT 算子）识别所有与其直接交互的变量，形成一个粗糙联合体 X。
   - 将 X 存入 N，并从 V_1 中移除 X 中的所有变量。
   - 调用扩展的交互探测（INTERACT-OV）扫描 V_1 中与 X 内变量可能存在重叠关系的变量，将其加入 OV。
   - 重复上述步骤直至 V_1 为空。
3. **第二阶段精炼**：
   - 对每个粗糙子组件 $X \in N$，SUD 算子构建其内部所有变量两两之间的交互强度矩阵，并进行连通分量分析。若某分量内的变量之间仅存在间接交互（通过矩阵分析可判定），则将该分量从 X 中拆出，作为新的独立子组件或与其他组件合并。
   - 对拆解后得到的候选重叠变量（即那些与多个子组件都有交互的变量），SD 算子进行多尺度扰动校验：在不同的扰动幅度下重复差分探测，若该变量与多个子组件的核心变量均保持强交互，则将其标记为真正的重叠变量，并记录它所属的所有子组件。
4. **输出**：最终得到一组可能带有重叠变量的子组件划分。

8.1.3 理论支撑

OEDG 的理论基础在于区分直接交互与间接交互。设三个变量 x_i, x_j, x_k，若 f 可写为 $f = g(x_i, x_k) + h(x_j, x_k)$ 的形式，则 x_i 与 x_j 之间不存在直接交互（$\partial^2 f/\partial x_i \partial x_j = 0$），但通过 x_k 它们会产生间接关联。在差分探测中，直接交互表现为二阶差分非零，间接交互则需通过三阶及以上差分来识别。SUD 算子正是通过分析三阶差分（即同时扰动三个变量）来检测间接耦合：若 $\partial^3 f/\partial x_i \partial x_j \partial x_k \neq 0$，说明 x_i 和 x_j 的交互依赖于 x_k 的状态，暗示它们可能通过 x_k 间接相关。基于此，SUD 能够识别出由间接交互导致的伪联合体，并将其拆解。

8.1.4 局限性与适用范围

OEDG 的优势在于能够精确还原重叠结构，但其代价也极为高昂：

- **计算开销指数增长**：第二阶段需要对每个疑似重叠组进行稠密矩阵校验，最坏情况下探测次数与 $2^{|OV|}$ 成正比，当重叠变量数量稍多时，开销将无法承受。
- **静态假设**：算法假设重叠拓扑结构在优化过程中固定不变，无法适应变量关系随进化动态变化的环境。
- **适用场景**：适用于结构严格固定、评估成本相对低廉且对重叠拓扑还原精度要求极高的静态优化问题（如某些已知结构的工程优化）。

8.2 基于贡献度的协同进化（CBCC-O, 2020）

前文介绍的 OEDG 试图通过高精度探测彻底切分重叠变量，但这种“硬切割”策略面临指数级计算开销的瓶颈。那么，是否存在一种思路，不追求完美切割，而是通过动态调度计算资源来“绕过”重叠问题呢？这正是 CBCC-O 的核心思想。

CBCC-O（Contribution-Based Cooperative Co-evolution for Overlapping Problems）由 Omidvar 等人于 2020 年提出，它坦然承认重叠结构难以完美切分，转而将问题转化为“时间域的动态资源博弈”——即在不同的进化阶段，将更多的计算资源（适应度评估次数）分配给当前对整体优化贡献最大的子组件，使其暂时获得对共享变量的主导权，从而隐式地处理重叠。

8.2.1 设计思想与技术突破

CBCC-O 基于一个简单观察：在包含重叠变量的问题中，不同子组件对全局目标改善的贡献往往是动态变化的。某个子组件可能在某一阶段迅速提升整体解的质量（例如，调整传动效率相关的变量带来显著收益），而另一子组件则陷入停滞。因此，与其静态固定资源分配，不如动态地将更多评估预算倾斜给当前“贡献大”的子组件，让它们优先优化共享变量。同时，为防止某些子组件长期被忽视而丧失多样性，当停滞的子组件偶尔获得资源时，对其执行“重多样化”操作，注入新鲜血液。

这里需要明确几个概念：

- **贡献（Contribution）**：指子组件在最近几代中对全局最优解改进的贡献程度，通常用最优值的变化量来衡量。
- **共享变量（Shared Variables）**：即前文所述的重叠变量，同时属于多个子组件。
- **停滞子组件（Stagnant Subcomponent）**：指长时间未能提升全局最优值的子种群，可能因多样性丧失而陷入局部最优。
- **历史贡献池（Historical Contribution Pool）**：记录每个子组件在过去一段时间内的贡献值，用于资源分配决策。

8.2.2 操作流程

1. **状态监测**：每轮进化后，监测所有子种群的演化轨迹。根据最近若干代是否产生更优解，将子种群划分为“活跃”（近期有贡献）和“停滞”（长期无改进）两类。
2. **贡献期望计算**：对每个子组件 i，计算其在最近两次迭代中对全局最优值的边际贡献：

$$\delta_i^{(t)} = f_{\text{best}}^{(t)} - f_{\text{best}}^{(t-1)}$$

   其中 $f_{\text{best}}^{(t)}$ 是第 t 代全局最优适应度值。若 $\delta_i^{(t)} > 0$，则更新该子组件的贡献记录；否则，沿用历史贡献的衰减值（例如乘以一个衰减因子），以平滑短期波动。
3. **动态资源倾斜**：对于每一个共享变量（即被多个子组件共同拥有的变量），考察所有包含该变量的子组件，从中选择贡献期望最大的子组件，为其分配额外的进化代数（即更多的 FE 预算）。这意味着该子组件将在下一轮获得更多机会优化共享变量，从而主导其取值。
4. **重多样化**：当某个被判定为“停滞”的子组件因历史权重而被选中获得资源时（即尽管它近期无贡献，但其历史贡献较高），算法会对其执行强制重多样化操作——例如随机重置部分个体、提高变异率、引入新个体等，以帮助该种群跳出局部最优，重新获得探索能力。

8.2.3 理论支撑

CBCC-O 的资源分配机制可视为一种基于贪心策略的多臂老虎机（Multi-armed Bandit）近似。在多臂老虎机问题中，玩家需要在多个选项（臂）中反复选择，以最大化累积收益，每个臂的收益分布未知。CBCC-O 将每个子组件视为一个“臂”，其收益定义为对全局最优的边际贡献 δ_i。算法采用“选择当前收益期望最高的臂”的贪心策略，并辅以重多样化机制来探索停滞的臂。该框架无需任何前置分解开销，其调度决策完全基于优化过程中自然产生的适应度变化信息，因此计算效率极高。

8.2.4 局限性与适用范围

CBCC-O 的主要局限在于资源分配依赖于滞后更新的历史贡献池。当重叠变量的物理重要性在短时间内剧烈漂移（即某一阶段至关重要的变量在下一阶段变得无关紧要）时，基于历史贡献的调度可能反应迟缓，导致资源错配。此外，对于共享变量高度耦合、各子组件贡献极度不平衡的场景，该策略效果显著；但如果各子组件贡献均衡且频繁交替，则资源调度的优势可能被削弱。

因此，CBCC-O 适用于存在大量难以拆解的静态重叠、且各子组件对全局目标贡献呈现明显不平衡特征的大规模优化问题。

8.3 分层差分分组（HDG, 2024）

前文介绍的 OEDG 和 CBCC-O 分别从“硬切割”和“软调度”两个角度处理了变量重叠问题。然而，真实系统的复杂性不仅体现在变量可能属于多个子组件（重叠），还体现在变量之间存在显著的地位差异——即**层次结构**。如果说重叠结构是变量关系的“横向”交织，那么层次结构就是“纵向”的主从控制。

在深入 HDG 之前，我们先理解什么是层次结构。考虑一个航空发动机设计问题：压气机的级数（记为变量 x_c）决定了发动机的整体构型和功率范围，属于“核心变量”；而每一级叶片的倾角（记为 $x_{e1}, x_{e2}, \ldots$）仅在给定级数下进行局部微调，属于“边缘变量”。核心变量的变化会引发适应度景观（即目标函数随变量变化的整体地形）的剧烈震荡，而边缘变量的影响则局限在局部区域。传统平权分组将核心变量与边缘变量等同对待，不仅浪费计算资源，更可能掩盖问题的本质结构——因为核心变量之间往往存在强交互，而边缘变量只与特定的核心变量相关。

这与前文的重叠问题形成鲜明对比：在重叠结构中，变量是“身兼数职”的共享资源；在层次结构中，变量是“发号施令”与“执行操作”的主从关系。OEDG 和 CBCC-O 均无法处理这种纵向的主从差异，因为它们将所有变量视为平权个体，只关注交互的存在与否，而不区分交互的“重要性层级”。

HDG（Hierarchical Differential Grouping）由 Liang 等人于 2024 年提出，旨在揭示这种“主从控制”关系，构建具有层次的分解结构，即先将核心变量聚合成主干，再将边缘变量挂载到相应的核心节点上，形成一棵嵌套树。这种“先主干后枝叶”的策略，与 CBCC-O 的“资源调度”思路不同，它追求的是对问题结构的深度解析，而非动态绕行。

8.3.1 设计思想与技术突破

HDG 的核心思想分为三步：

1. **区分核心与边缘**：通过对变量施加不同幅度的扰动，识别哪些变量能引起全局性变化（核心），哪些仅产生局部影响（边缘）。这一步本质上是给变量“定级”。
2. **梳理主干拓扑**：暂时忽略边缘变量，只在核心变量空间中运行高效的记忆二分搜索（类似 EADG），识别核心变量之间的交互关系，形成主干网络（即哪些核心变量应该被分在同一子组件）。这一步复用了前文 EADG 的高效探测思想，但仅限于核心变量。
3. **深度挂载边缘变量**：将每个边缘变量通过局部探测，挂载到与之交互最强的核心变量上；若某个边缘变量与多个核心变量均有交互，则标记为跨层次共享变量（这种变量在层次结构中处于特殊位置，可视为“重叠”与“层次”的复合体）。

这种设计使得 HDG 既能揭示层次关系，又能兼容前文处理过的重叠情形——跨层次共享变量正是重叠变量在层次框架下的表现形式。

8.3.2 操作流程

1. **核心-边缘非对称推演**：

- 对每个变量 x_i，分别施加小幅度扰动 δ（例如 10^{-6} 倍变量范围）和大幅度扰动 Δ（例如 10^{-2} 倍变量范围）。
- 记录小扰动下的局部变化量 $\Delta_{\text{local}} = |f(\mathbf{x} + \delta\mathbf{e}_i) - f(\mathbf{x})|$ 和大扰动下的全局变化量 $\Delta_{\text{global}} = |f(\mathbf{x} + \Delta\mathbf{e}_i) - f(\mathbf{x})|$。
- 若 $\Delta_{\text{global}}/\Delta_{\text{local}} > \theta$（$\theta$ 为阈值，如 10），则认为该变量在大尺度上能引发显著变化，标记为**核心变量**；否则标记为**边缘变量**。

2. **空间缩减与主干拓扑识别：**
   - 暂时将所有边缘变量从变量集中移除，仅在核心变量集合上运行 EADG 风格的记忆二分搜索，识别核心变量之间的交互关系，形成若干核心子组件（称为“主干”）。这一步利用了“核心变量主导全局趋势”的假设，认为边缘变量的扰动在主干识别阶段可视为可忽略的噪声。
   - 输出核心变量之间的分组结果，以及每个核心子组件的代表变量。
3. **深度挂载边缘变量：**
   - 对于每个边缘变量 x_e，依次考察其与各个核心子组件的交互强度。具体做法：固定其他核心变量的取值，分别扰动 x_e 并观察目标函数变化，同时检测 x_e 的效应是否随核心组件内变量的变化而改变。
   - 将 x_e 挂载到交互强度最大的核心子组件上。若 x_e 与多个核心子组件均有显著交互，则将其标记为跨层次共享变量——这实际上就是前文 OEDG 所追求的重叠变量，但在这里它被置于层次框架下理解。
   - 最终形成一棵以核心变量为根节点、边缘变量为叶节点的层次树。

8.3.3 理论支撑

HDG 的理论基础源于**变量敏感性分析**。定义变量 x_i 的敏感性指标为：

$$s_i = \sup_{\|\Delta\mathbf{x}\| \le \epsilon} \frac{|f(\mathbf{x} + \Delta\mathbf{x}) - f(\mathbf{x})|}{\|\Delta\mathbf{x}\|},$$

即单位扰动下目标函数的最大变化率。核心变量通常具有较大的 s_i（因为小扰动就能引起大变化），而边缘变量的 s_i 较小。通过在不同尺度上施加扰动，可以近似估计 s_i 并区分核心与边缘。

空间缩减的可行性基于以下观察：在优化问题中，核心变量决定了适应度景观的宏观走势，边缘变量的局部扰动不会改变核心变量之间的交互关系。因此，在识别核心变量分组时暂时忽略边缘变量，不会导致主干拓扑的错误——这与 CBCC-O“资源向贡献大的子组件倾斜”的思路有异曲同工之妙，都是聚焦于“重要”的部分。

记忆二分搜索复用 EADG 的理论，确保核心变量交互识别的效率。而边缘变量的挂载过程本质上是检测“变量-子集”交互，可通过多尺度扰动（即逐步扩大扰动范围）来验证归属，其正确性建立在“若边缘变量与核心变量强相关，则其效应会随核心变量状态改变而改变”的直觉上。

8.3.4 局限性与适用范围

HDG 的优势在于能够揭示变量间的主从控制关系，构建层次化分解结构，这对于理解复杂工程系统的内在机理具有重要意义。然而，它存在一个严重的**计算悖论**：为了获得高保真的层次结构，必须在优化初期投入大量的探测成本（包括核心-边缘区分和多轮递归探测），这些成本可能超过后续进化所能节省的算力，导致“得不偿失”——这与 OEDG 面临的困境相似，都是“精度换效率”的体现。

因此，HDG 更适用于具有明确层次结构、且对计算预算相对宽松的理论研究或特定工程问题（如系统辨识、结构分析），而非通用的在线优化场景。它代表了与 CBCC-O 完全不同的技术路线：前者追求对问题结构的深度解析（哪怕付出高昂代价），后者则通过动态调度绕过结构解析的困难。

9 学习型分解与古典融合的范式转移（2024-2026）

2024 年后，研究路线发生剧烈分化：一条是由深度强化学习主导的颠覆性“动态学习型分解”，另一条则是古典分解理论在多级管线架构下的“折中与坚守”。

9.1 基于学习的协同进化（LCC, 2025）

前文介绍的各类算法——无论是基于数学探测的 EADG、DDG、OEDG、HDG，还是基于资源调度的 CBCC-O——都依赖于手工设计的规则（如差分阈值、贡献计算公式）或固定的启发式策略。尽管它们在各自目标上取得了突破，但面对千变万化的未知问题结构，这些“死板”的规则难免力不从心。那么，能否让算法自己学会“如何分解”呢？这正是 LCC（Learning-based Cooperative Coevolution）的出发点。

LCC 由 Chen 等人于 2025 年提出，它将问题分解策略的选择建模为一个**序列决策过程**，通过**离线训练**的强化学习智能体，在**在线执行**时动态调度分解策略。可以把 LCC 的智能体想象成一个游戏玩家：每一代的种群状态就是“游戏画面”，选择哪种分解策略就是“按下手柄按钮”，优化带来的效果提升就是“游戏得分”。而**近端策略优化（PPO）**就是一套训练玩家如何根据画面、通过试错来学会获得高分的方法。这里的“离线训练”指在优化开始前，先用大量已知的优化问题（称为**元问题**）对智能体进行预训练；“在线执行”则指在具体的新问题上直接应用训练好的模型，无需再消耗评估次数进行探测。

9.1.1 设计思想与技术突破

LCC 的核心创新在于将分解策略的动态选择转化为马尔可夫决策过程（MDP），从而将机器学习引入协同进化。智能体通过实时提取的 **58 维状态特征向量**进行**前向推理**（即神经网络计算输出，仅需毫秒级时间，不消耗任何真实适应度评估），输出策略池中三种分解动作的概率分布，实现了在线零额外评估开销。

9.1.2 状态空间设计

58 维特征向量由三类信息拼接而成，旨在全面刻画当前优化进程的状态：

- **全局优化状态特征（12 维）**：包括种群适应度的均值、方差、偏度、收敛斜率（最近几代最优值的变化趋势）、最优值停滞代数（连续未改进的代数）等。这些特征反映了宏观优化态势，例如是否陷入局部最优、收敛速度快慢。
- **子组独立动态特征（$4 \times m$ 维）**：对当前划分出的 m 个子组（m 由上一轮的分解策略决定），每个子组计算 4 个统计量：
  - 组内变量的协方差矩阵对角线分布（可理解为变量自身方差的平均值，反映组内变量波动幅度）；
  - 组内个体的平均适应度；
  - 组内多样性指标（如个体间平均距离）；
  - 该子组当前最优解与全局最优解的欧氏距离。

  协方差矩阵可以简单地理解为描述变量之间“步调一致性”的表格——如果两个变量总是同增同减，说明它们很可能属于同一个子问题。
- **历史动作上下文特征（$2 \times L$ 维）**：记录最近 L 代（L 通常取 10）中每种分解策略的使用频次和累积收益（即采用该策略后带来的适应度提升之和），以提供决策的历史记忆。

这些特征共同构成了 58 维向量，作为智能体决策的依据。

9.1.3 动作空间与奖励函数

智能体的动作空间包含三种候选分解策略，每种策略对应一种变量分组方式：

- **随机分解（RD）**：将变量随机划分为大小相等的子组。这种策略有助于跳出局部最优，在种群陷入停滞时重新组织搜索方向。
- **最小方差分解（MiVD）**：基于当前种群中变量的协方差矩阵，将相关性强的变量划分到同一子组。这有助于精细收敛，因为强相关变量往往需要协同优化。
- **最大方差分解（MaVD）**：将相关性弱的变量分到同一子组，以维持探索多样性，防止种群过早同质化。

在每一代，智能体根据状态输出选择其中一种动作执行。

奖励函数 r_t 定义为：采用动作后，底层优化器（如 CMA-ES）在有限预算（通常为总 FE 预算的 10%）内实现的累积适应度提升。奖励越高，说明该动作对优化进程越有利。

9.1.4 离线训练与在线执行

智能体采用**近端策略优化（PPO）**算法进行离线训练。训练数据集由大量**元问题**构成，包括 CEC 基准函数族、随机生成的 Wmodel 问题（一种可控制变量交互类型的测试问题）等。每个元问题都是一个完整的优化问题，智能体在其上反复尝试不同的分解策略，根据获得的奖励更新神经网络参数，逐步学会在不同状态下选择最优动作。

训练完成后，智能体即可用于在线执行。在每一代优化中，算法实时提取 58 维状态特

征，输入训练好的神经网络（三层 MLP）进行前向推理，输出三种动作的概率，然后按概率采样执行具体的分解策略。整个过程不消耗任何额外的适应度评估（仅涉及计算已有点的统计量），因此在线开销极低。

9.1.5 理论支撑与局限性

LCC 的理论基础源于强化学习的马尔可夫决策过程（MDP）框架和 PPO 的策略优化理论。状态特征的设计借鉴了探索性景观分析（Exploratory Landscape Analysis, ELA）的成熟成果——ELA 旨在从少量采样中提取优化问题的可量化特征，用于预测算法性能。

然而，LCC 也面临严峻挑战：

- **泛化风险**：离线训练建立在“训练集分布能覆盖未来问题”的假设上。当遇到与训练数据分布显著不同的全新问题（即 OOD，Out-of-Distribution）时，智能体的决策可能完全失效，甚至导致性能急剧下降。
- **可解释性丧失**：智能体的决策依据隐藏在神经网络数百万个权重中，无法提供数学解释——即算法做出分解决策的依据不明，我们无法理解它为什么将某些变量分在一起。这导致在关键工程领域（如航天设计）中，工程师不敢信任黑箱给出的结果。
- **训练成本**：离线训练需数万 GPU 小时，数据收集需数百万次元问题评估，普通研究团队难以承受。

因此，LCC 适用于动态未知结构、对可解释性要求不高、且拥有足够离线训练资源的场景。它与前文所有基于数学探测的算法形成了鲜明对比：后者追求“理解问题再分解”，而 LCC 追求“学会如何分解”。这种范式转移标志着协同进化算法正从“手工设计”走向“数据驱动”。

9.2 复合可分性分组（CSG, 2026）

前文介绍的算法各有所长：EADG 高效处理加性可分问题，DDG 扩展至乘性结构，OEDG 和 HDG 分别攻克重叠与层次复杂性，而 LCC 则开辟了学习型分解的新范式。然而，真实世界的问题往往是多种结构的复合体——既包含加性分量，又包含乘性耦合，还可能存在局部非可分纠缠。单一策略难以全面应对。CSG（Composite Separability Grouping）由 Liu 等人于 2026 年提出，旨在通过一个多阶段逐步过滤的框架，将上述多种检测手段有机整合，实现对复杂混合结构的系统性解析，堪称古典分解理论的集大成者。

9.2.1 设计思想与技术突破

CSG 的核心思想是“分而治之，逐层剥离”：先处理最容易识别的结构，逐步缩小问题规模，最后攻坚最难的非可分部分。它构建了一个四阶段管线：

1. **加性可分变量检测**：首先使用经典 DG（即前文所述的基础差分分组）快速扫描所有变量，识别并剥离纯加性可分变量（即那些可以独立优化的变量组）。这一步利用 DG 的高效率，大幅缩减后续需要精细探测的变量数量。

2. **乘性可分变量检测（MSVD）**：针对剩余变量中可能存在的乘积结构，调用改进的乘性检测算子（MSVD）。与 DDG 不同，MSVD 避免了对数变换带来的数值风险，采用“比值扰动法”：比较 $f(a,b)/f(c,b)$ 与 $f(a,d)/f(c,d)$ 的差异，通过比值的稳定性判断乘性可分性。这种方法对函数值域没有非负要求，数值鲁棒性更强。
3. **一般性可分变量检测（GSVD）**：经过前两步，剩下的变量往往具有复杂的非典型可分性（例如混合加性与乘性，或依赖特定区域）。此时调用基于“最小点偏移原理”的检测算子 GSVD。该原理指出：在局部极小值附近，如果变量是可分的，那么单独扰动该变量不会引起其他变量最优值位置的偏移；反之，如果变量与其他变量耦合，则扰动会迫使其他变量的最优值发生移动。算法在局部极小值附近执行黄金分割搜索（一种一维搜索方法，通过不断缩小区间找到极值点），并通过单点对集合（one-to-all）的方式——即每次只扰动一个变量，观察所有其他变量的响应——来检测这种偏移，每个维度约消耗 40 次 FE 即可完成校验。
4. **非可分变量分组（NVG）**：经过前三层过滤，最后剩下的变量是极度纠缠、难以直接解析的非可分变量。此时调用递归分组检测（RGD）。RGD 采用二分法思想：向候选变量池施加逐步增大的联合扰动，一旦发现适应度出现非零突变（表明池内存在交互），便将变量池对半切分，对两个子池递归调用同样的检测，直至定位到具体的交互组。最坏情况下，探测次数为 $O(k\log n)$，其中 k 为非可分变量组的数量。

这种逐级过滤的设计，使得 CSG 既能利用 DG 的高效性处理简单结构，又能通过 GSVD 和 RGD 覆盖复杂的非线性耦合，将通用非线性解析的成本控制在工程可接受的范围内（即“拉回工程可用红线”）。

9.2.2 理论支撑

CSG 的数学基础建立在对不同类型可分性的严格定义上：

- **加性可分**：函数可表示为多个子函数之和，且子函数变量互不相交。
- **乘性可分**：函数可表示为多个子函数之积。
- **一般可分**：指变量在局部邻域内对其他变量的最优值无影响，即满足“最小点偏移原理”。

其中，“最小点偏移原理”的数学依据是：在局部极小值点 $\mathbf{x}^*$ 附近，若变量 x_i 与 x_j 相互独立，则对于任意小扰动 δ，固定 $x_i = x_i^* + \delta$ 后，x_j 的新最优值仍为 x_j^*（或变化极小）；反之，若两者耦合，则 x_j 的最优值会发生显著偏移。GSVD 通过黄金分割搜索精确找到当前最优值，再扰动目标变量并重新优化其他变量，从而定量测量这种偏移。

RGD 的递归逻辑基于“变量集整体响应”的思想：若一个变量集整体受到扰动后函数值变化与各变量单独扰动之和不同，则说明集内存在交互；通过二分递归，可以定位到具体的交互对。这一方法保证了在最坏情况下仍能有效识别非可分组，且复杂度可控。

9.2.3 局限性与适用范围

CSG 作为古典分解理论的集大成者，保持了数学上的可解释性和理论保证。然而，它仍受限于单变量绝对梯度置信度的基本假设——即依赖局部扰动信号来判断交互，在高度欺骗

性函数（即函数景观故意引导算法向错误方向搜索，例如最优解周围全是更差的解，让算法误以为到了山顶，其实只是个小山包）或强噪声环境中，这些信号可能被淹没，导致失效。此外，逐步过滤机制可能导致早期阶段的误差向后传递（例如，第一阶段误判的变量会影响后续所有检测）。因此，CSG 适用于复合非线性可分、对理论保证和安全逻辑透明度有严苛要求的工程与科研场景，如航空航天系统设计、复杂物理模型校准等。

10 其他值得关注的新方法与预研趋势（2023-2026）

在前文介绍的主流算法之外，还有一些富有启发性的探索方向，它们虽未形成完整体系，但代表了未来可能的突破点，构成了下一代协同进化算法的理论前沿。

10.1 基于变量重要性的动态协同进化（DCCVI, 2025）

前文介绍的许多算法（如 EADG、OEDG、HDG）都试图通过尽可能多的探测来完整还原问题的变量交互图，这种思路可称为“静态全量拓扑解析”——即在优化开始前或初期，就试图弄清楚所有变量之间的关系。然而，对于超大规模问题，这种追求“全量”的做法往往代价高昂，且许多细节信息在优化后期可能变得无关紧要。

DCCVI（Dynamic Cooperative Coevolution based on Variable Importance）由 Zhang 等人于 2025 年提出，它反其道而行之：放弃追求全量解析，转而聚焦于那些对优化贡献最大的“精英变量”。其核心思想分为两步：

- **启动阶段：收集边际贡献数据**。首先使用廉价的随机分组进行多轮优化，记录每个变量对全局最优值提升的贡献程度，即“边际贡献”——指当该变量被优化时，全局最优值改善的幅度。这一策略称为 EDC（Elite Contribution Collection）。
- **动态分组阶段：构建专属动态子组件**。根据积累的贡献数据，筛选出贡献率最高的若干变量（称为精英变量），将它们组成一个特殊的动态子组件，并对其分配大量的计算资源进行“算力饱和攻击”——即集中大部分评估预算专门优化这些关键变量。同时，随着优化进程，精英集合会动态调整。

此外，DCCVI 还引入了“阶段性参数自适应策略”（SSPA，Stage-wise Self-adaptive Parameter Strategy）：每当子组件的变量结构因精英集合变化而重组时，底层演化算法（如差分进化）的控制参数（如变异率、交叉率）也会自动调整，以适应新的子问题特征。

该工作的重要贡献在于：它证明了在非可分黑盒问题（即变量间存在复杂交互且无结构先验的问题）中，基于重要性的动态精英集结策略，比试图解析所有变量关系的全量方法具有更高的寻优效率。

10.2 基于图神经网络的分解（GNN-based Decomposition）

将变量交互关系自然地建模为图结构：每个变量是图中的一个节点，变量间的交互是节点之间的边，整个问题的依赖关系就构成了一张“交互图”。基于图神经网络（GNNs）的分解

方法试图离线学习这种图的模式——即利用大量已知问题（如 CEC 基准测试函数，CEC 是进化计算领域常用的标准测试函数集）的真实交互图作为监督信号，训练 GNN 模型，使其能够从部分采样（例如仅探测 10% 的变量对）中预测出完整的全局交互图。

当前的研究进展表明，在训练集覆盖的问题类型上，该方法可以达到 85% 以上的预测准确率。但主要挑战在于泛化性：当遇到训练中从未见过的新型函数结构时，预测精度急剧下降。此外，可解释性也是难题：GNN 的预测结果缺乏数学证明，我们无法确信其给出的交互图是否正确。

10.3 时空图持续跟踪（Spatiotemporal Graph Continuous Tracking）

真实世界中的优化问题往往具有动态性——变量间的依赖关系并非一成不变，而是随着搜索进程演化（例如，在优化初期重要的全局变量，到后期可能退化为局部微调变量）。然而，现有的大多数分解方法（如 OEDG）都假设交互关系是静态的，一旦分解完成就不再改变。

时空图持续跟踪引入时空图神经网络（STGNNs）的思想，将进化过程中的每一次上下文向量更新（即每一代找到的新解）视为问题依赖图谱的一次微小“时间切片”。通过分析适应度轨迹（即历代最优解的变化路径）在时空图谱上引起的局部拓扑震荡信号，算法可以实时感知哪些变量之间的关系正在发生变化，并平滑地微调重叠边界，而不是每隔若干代就进行一次代价高昂的全局重分解。这一方向有望将原本集中爆发的分解开销，像涓涓细流一样平滑地分摊到整个进化周期，极大提升计算鲁棒性。

10.4 多臂老虎机资源感知调度（Multi-armed Bandit Resource-Aware Scheduling）

不同分解算子的“保真度”（即解析精度）和计算开销各不相同：一维分解几乎不消耗额外评估，但完全忽略交互；EADG 以 $O(n \log n)$ 的代价捕捉加性结构；而 OEDG 虽能精确还原重叠，但开销指数级增长。在实际优化中，我们并不总是需要最高保真度的分解——初期可能粗粒度就足够，后期才需要精细切割。

多臂老虎机资源感知调度将不同保真度的分解算子视为多臂老虎机问题中的不同“臂”。通过置信上界（UCB）算法，控制器在线学习每个臂的预期收益（收益定义为每单位适应度评估带来的性能提升）。同时，控制器实时感知当前种群的内部状态（如多样性、停滞窗口长度、剩余预算），智能判断何时应该进行粗粒度探索、何时需要动用高保真切分手术刀。这一方向将“分解精度”本身作为一个可动态优化的超参数，以经济学的视角平衡探测成本与进化收益，从根本上应对“分解开销过大”的世纪难题。

11 横向对比与元分析

为深刻理解各类算法的演进逻辑，本节从开销-精度权衡、问题类型覆盖、假设脆弱性、理论-实践割裂四个维度进行横向元分析。

11.1 开销-精度权衡曲线

问题分解技术的迭代本质上是多维零和博弈：没有算法能同时占据“高在线效率、高解析精度、低离线成本”的完美帕累托最优前沿。表 1 和表 2 总结了各算法的关键属性与性能权衡。

表 1: 核心问题分解算法的关键属性对比

算法	年份	归类	额外在线 FEs 开销	理论复杂度（在线）
EADG	2022	基于变量交互的静态分解	是（显著降低）	$O(n \log n)$
DDG	2022	基于变量交互的扩展分解	否（复用加性探测）	$O(D^2/m)$
CBCC-O	2020	基于资源调度的动态机制	否（绝对零前置）	$O(m)$（线性轮询）
OEDG	2024	高精度静态交互精炼分解	是（极高）	随重叠密度指数飙升
HDG	2024	层次化静态交互剖析分解	是（极高）	$O(2D + D \log k + G \log k)$
LCC	2025	数据驱动的动态学习型分解	否（在线为零，离线巨大）	$O(1)$（前向推理）
CSG	2026	逐步精炼的静态综合交互分解	是（中等）	介于 $O(n \log n)$ 与 $O(n^2)$ 之间

11.2 问题类型光谱的覆盖边界

从完全可分到动态未知结构，各类算法的历史坐标可标定为：

$$\text{完全可分} \xleftrightarrow{\text{EADG}} \text{加性可分} \xleftrightarrow{\text{DDG/CSG}} \text{乘性/复合可分} \xleftrightarrow{\text{CBCC-O/OEDG}} \text{静态局部重叠} \xleftrightarrow{\text{LCC}} \text{动态未知流变结}$$

DDG 和 CSG 成功将古典算法边界推进至乘性和复合可分区域，但在高密度重叠网络面前力不从心。OEDG 和 CBCC-O 首次系统覆盖“静态重叠”，但付出了巨大开销代价。LCC 在表面上覆盖全光谱，但其实现依赖于宏观启发式规则的粗糙切换，而非微观数学解析。

11.3 假设的隐蔽性与脆弱性

每个强大算法背后都隐藏着可能导致崩溃的先验假设：

表 2: 核心问题分解算法的性能权衡对比

算法	拓扑结构覆盖光谱	结构识别精度特征	理论保证与可解释性	主要局限与隐患风险
EADG	加性可分、部分可分	宏观高（弱交互模糊，误差沿树传播）	强（基于差分定理传递性）	弱交互误判，误差雪崩式传播
DDG	加性结构、乘性非线性耦合	高（扩展了乘法识别，但数值稳定性脆弱）	中等（受限定义域）	对数变换引发数值下溢与梯度灾难
CBCC-O	静态局部重叠、不平衡贡献网络	基于调度规避硬切割，容错率高（但滞后）	弱（贪心策略近似）	静态贡献度池对动态权重漂移反应滞后
OEDG	极度纠缠交叠拓扑、伪联合体	极高（精确还原平面重叠，不适应动态）	强（基于高阶交互分析）	指数级开销，无法适应动态环境
HDG	深度层次嵌套、主从控制流	极高（解析层次结构，但计算悖论严重）	极强（基于敏感性分析）	为证明边缘变量消耗海量无效探测算力
LCC	动态流变、完全未知黑盒结构	策略调度级（依赖训练数据，OOD 风险大）	极弱（纯黑盒）	OOD 泛化失效与灾难性策略坍塌
CSG	复合非线性可分、一般交互模式	高（整合 DG 效率与 GSG 广度）	强（基于最小点偏移原理）	受限于单点偏导可靠性假设

- **差分变种体系（EADG/DDG/HDG/CSG）**：假设“单变量微小扰动引起的适应度绝对变化是检测交互唯一可靠的信号”。在欺骗性函数（指函数景观故意引导算法向错误方向搜索）、极端多模态或强噪声景观中，该假设失效。
- **重叠处理体系（OEDG/CBCC-O）**：假设重叠拓扑结构在进化周期中静态不变。一旦环境动态变化，昂贵的预分解瞬间失效。
- **学习型体系（LCC）**：假设深度强化学习模型的泛化性理所当然。在陌生黑盒环境中，缺乏符号约束的 Agent 极易发生灾难性策略坍塌。

11.4 理论保证与工程实效的割裂

EADG、HDG、CSG 等算法努力维持演化计算作为应用数学分支的尊严，提供严密的复杂度上界推导与透明执行逻辑。而以 LCC 为首的机器学习派系则将问题分解变为“神经网络架构调参”和“奖励函数工程”的实验炼金术。这种理论与实践的深度割裂，标志着协同进化算法正处于可能完全失去数理根基的历史十字路口。

12 未来展望

基于对过去七年技术演进的深刻解构，单纯修补成对扰动公式的古典改良路线潜力已尽，而完全依赖黑盒强化学习的极端路线也非坦途。未来三到五年，以下三个方向有望实现突破。

12.1 方向一：神经引导与符号推演的混合架构（Neuro-Symbolic Decomposition）

未来的分解器不应被迫在“死板的专家规则”与“不透明的黑盒神经网络”之间二选一。神经符号混合系统旨在实现数理严谨与数据驱动的优势互补：

- **离线训练**：利用 GNN 或大语言模型学习海量优化问题的元特征和高阶拓扑模式，构建结构先验知识库。
- **在线推理**：神经模型输出柔性结构先验——例如，一个连续的概率邻接矩阵，指示任意变量对存在交互的潜在概率。
- **符号推演**：将概率图谱注入轻量级符号分组算法（如改进的 EADG 或 CSG），在强先验引导下进行确定性的逻辑推演与边界切分。

符号算法可通过数学定理（如传递性、对称性）对 GNN 的概率输出进行校验与修正，从而守住可解释性底线。主要挑战在于设计 GNN 的输出表征，使其既能被符号算法理解，又能保留概率信息。

12.2 方向二:针对动态重叠环境的时空图持续跟踪(Spatiotemporal Graph Continuous Tracking)

当前重叠检测技术受限于“静态快照”思维，预设 $T = 0$ 时刻切分的架构在 $T = 1000$ 代时依然适用。然而，真实动态优化中变量关系随进化过程演化。未来应引入时空图神经网络（STGNNs）：

- 将每次上下文向量更新视为问题依赖图谱的一次微小时间切片演化。
- 通过捕捉适应度轨迹在时空图谱上引起的局部拓扑震荡信号，实时平滑微调重叠边界。
- 避免每隔数十代进行一次昂贵且破坏种群平衡的全局重分解。

时空图网络在交通预测领域已成熟，迁移至进化计算需解决适应度轨迹的图编码问题以及噪声干扰下的稳健跟踪问题。预期突破在于将集中爆发的分解开销平滑分摊到整个进化周期。

12.3 方向三：资源感知的自适应保真度分解（Resource-Aware Adaptive Fidelity Decomposition）

当前算法陷入“盲目追求最高切割精度”的技术执念，忽略“过犹不及”的资源论真理。优化初期，粗糙廉价的分层（如一维分解）足以提供有效梯度；仅在深度停滞时才需动用高保真切分手术刀。未来应嵌入元资源控制核心：

- 将不同保真度的分解算子（从一维分解到 OEDG 级别）视为多臂老虎机的“臂”。
- 元控制器实时感知种群多样性、停滞窗口长度、剩余 FE 预算，通过 UCB 算法在线学习每个臂的预期收益。
- 动态决策当前应调用何种保真度的分解。

多臂老虎机已在 CBCC 的资源分配中有所应用，将其扩展至“分解保真度选择”是自然升级。核心挑战在于量化不同分解算子的预期收益（每单位 FE 带来的适应度提升）并处理收益估计的噪声。

12.4 三个方向的协同关系

这三个方向具有深刻的协同潜力：神经符号混合（方向一）可为自适应保真度（方向三）提供更精准的保真度评估依据——GNN 输出的概率邻接矩阵能反映当前分解的置信度；时空图跟踪（方向二）可为方向一提供动态演化的训练数据，使 GNN 学会时间演化模式；方向三的调度决策可反向指导方向二的跟踪焦点——当需要高保真分解时，激活时空图的局部高分辨率跟踪。三者的融合有望构建自适应、可解释且高效的下一代协同进化分解框架。

13 结语

协同进化算法自 1994 年诞生以来走过辉煌三十年，其问题分解技术在 2020 至 2026 年间经历了从加性数学推导（EADG, DDG）到重叠工程拓扑（OEDG, CBCC-O, HDG），再到

数据驱动范式（LCC, CSG）的剧烈蜕变。这段历史勾勒出清晰的演进轨迹：**精度竞赛 → 结构觉醒 → 范式革命**。

当前领域正处于“数据驱动的工程黑盒实效”与“古典逻辑的数理白盒严谨”的剧烈碰撞之中。机器学习带来的零在线开销神话，与系统理论可解释性的丧失之间产生的巨大张力，定义了未来十年的核心研究议程。可解释性危机必须被正视——当算法自主决定“如何分解”时，我们如何信任其决策？当优化失败时，我们如何追溯是分解策略还是搜索算法的问题？

我们呼吁下一代研究者正视这种危险的理论割裂，不要让协同进化这一优美框架沦为纯粹依赖调参的深度学习炼金术。所幸的是，神经符号混合等方向已经为我们指明了一条可能的出路：我们或许无需在“白盒”和“黑盒”之间做非此即彼的选择，而是可以用“黑盒”的直觉去引导“白盒”的推理，最终打造出既强大又可信的新型算法。

华南理工大学 SOUTH CHINA UNIVERSITY OF TECHNOLOGY 1918